Министерство образования Республики Беларусь
Учреждение образования «Белорусский государственный университет информатики и радиоэлектроники»

Факультет компьютерных систем и сетей
Кафедра Информатики
Дисциплина «Методы защиты информации»

ОТЧЕТ

к лабораторной работе №7

на тему:

«КРИПТОГРАФИЯ С ИСПОЛЬЗОВАНИЕМ ЭЛЛИПТИЧЕСКИХ КРИВЫХ»

БГУИР 6-05-0612-02 73

Выполнил студент группы 353504
ЛЕВШУКОВ Дмитрий Александрович

Проверил ассистент каф. Информатики
ПОЛЯКОВ Кирилл Борисович

Минск 2026

СОДЕРЖАНИЕ

Введение 3
1 Выполнение работы 4
Вывод 6
Приложение А (обязательное) Исходный код программы 7

ВВЕДЕНИЕ

Одним из направлений современной криптографии является криптография на основе эллиптических кривых. Эллиптическая кривая над конечным полем может быть задана уравнением вида.

$$y^2 \equiv x^3 + ax + b \ (mod\ p),$$

где a и b – параметры кривой, а p – простое число, определяющее конечное поле.

Точки, координаты которых удовлетворяют данному уравнению, используются в криптографических операциях. Для них определены операции сложения и умножения точки на целое число.

Криптографические алгоритмы на эллиптических кривых позволяют создавать схемы шифрования и обмена ключами. Одним из примеров является аналог алгоритма Эль-Гамаля, в котором для шифрования используются открытый ключ получателя и дополнительный случайный параметр. Получатель, располагающий закрытым ключом, может восстановить исходное сообщение.

Общий принцип работы аналога алгоритма Эль-Гамаля на эллиптических кривых заключается в следующем. Сначала выбираются параметры кривой и базовая точка G. Получатель задаёт закрытый ключ d и вычисляет открытый ключ $Q = dG$. При шифровании сообщение представляется точкой M на эллиптической кривой, после чего выбирается параметр k и формируется пара точек шифртекста: $C_1 = kG, C_2 = M + kQ$.

Для расшифрования получатель вычисляет dC_1, находит противоположную точку $-dC_1$ и складывает её с C_2. В результате восстанавливается исходная точка сообщения: $M = C_2 - dC_1$.

Задание:

1 Изучить теоретические сведения.

2 Реализовать схему шифрования (дешифрования) для аналога алгоритма Эль-Гамаля на основе эллиптических кривых.

1 ВЫПОЛНЕНИЕ РАБОТЫ

Общий алгоритм шифрования и расшифрования на основе эллиптических кривых представлен в виде блок-схемы на рисунке 1.1.

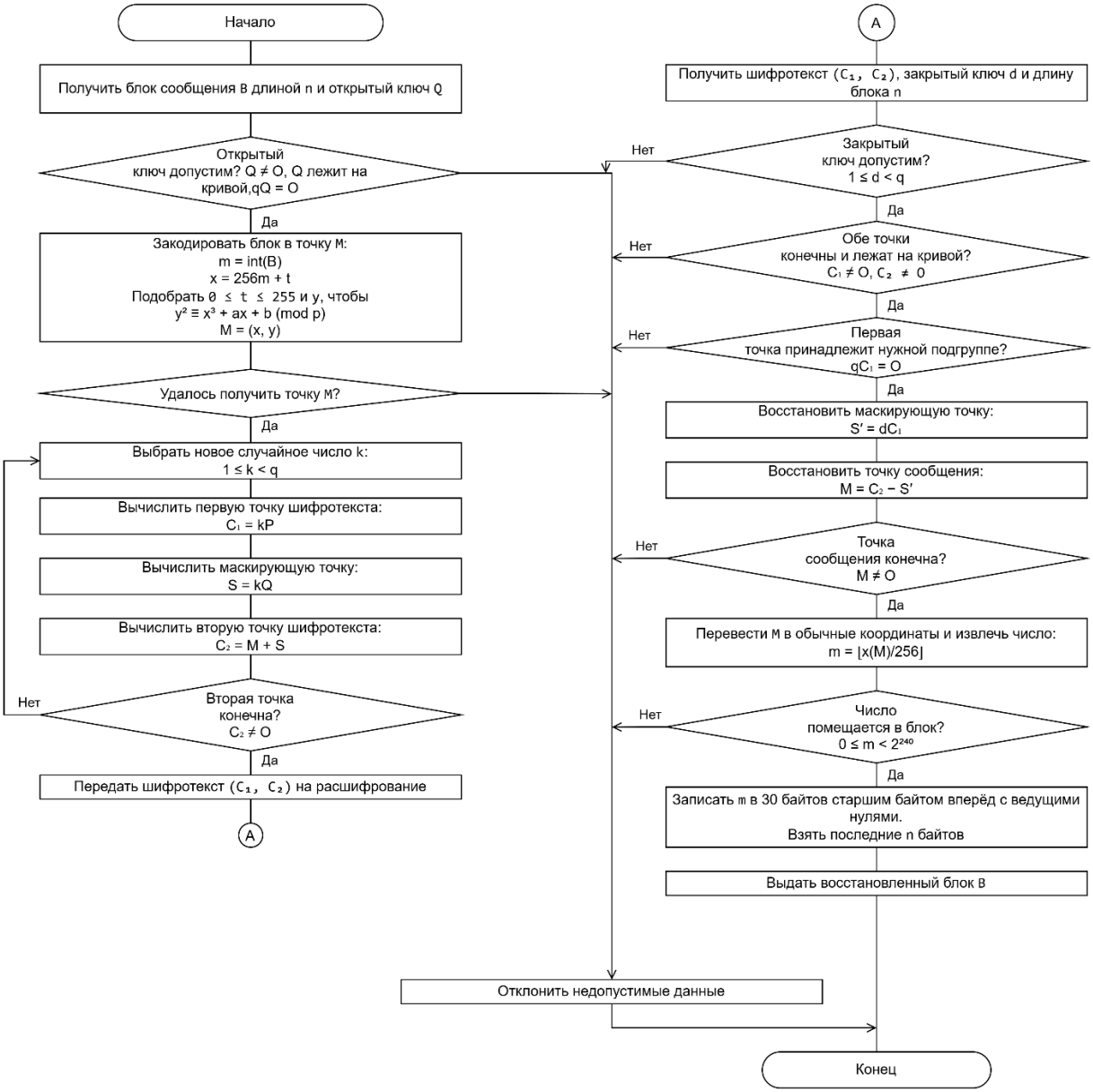


Рисунок 1.1 – Блок-схема алгоритма шифрования и расшифрования на основе эллиптических кривых

Результат выполнения программы в режиме генерации ключей представлен на рисунке 1.2.



Рисунок 1.2 – Результат выполнения программы в режиме генерации ключей

Результат выполнения программы в режиме шифрования представлен на рисунке 1.3.

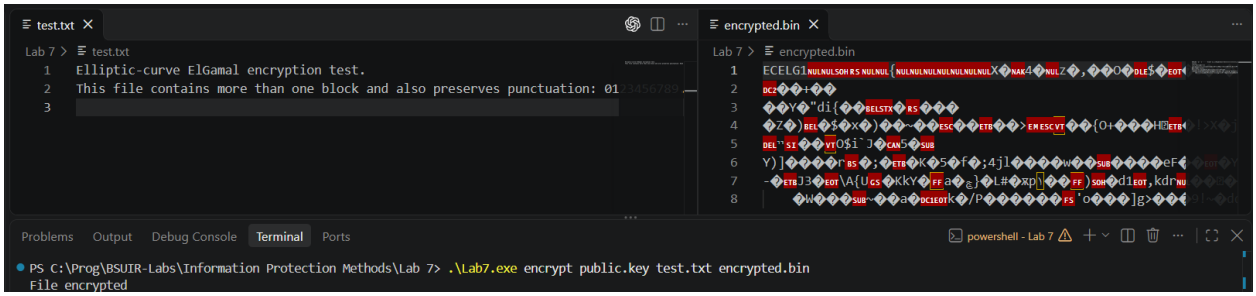


Рисунок 1.3 – Результат выполнения программы в режиме шифрования

Результат выполнения программы в режиме расшифрования представлен на рисунке 1.4.



Рисунок 1.4 – Результат выполнения программы в режиме расшифрования

ВЫВОД

В ходе лабораторной работы был реализован на языке C++ алгоритм шифрования Эль-Гамаля на эллиптических кривых. Изучены операции сложения и удвоения точек, умножения точки на целое число, а также принципы формирования закрытого и открытого ключей.

Разработанная программа выполняет генерацию ключей, шифрование и расшифрование файлов. При шифровании блоки исходных данных преобразуются в точки кривой, для каждого блока выбирается случайное одноразовое число и формируется пара точек шифротекста. При расшифровании с помощью закрытого ключа восстанавливаются точки сообщения и исходные байты.

Тестирование подтвердило побайтовое совпадение исходных и восстановленных данных, в том числе для пустого файла и неполного последнего блока. В результате работы закреплены практические навыки выполнения операций над точками эллиптической кривой и реализации асимметричного шифрования.

ПРИЛОЖЕНИЕ А
(обязательное)
Исходный код программы

```
#include "CipherFile.hpp"

#include <iostream>
#include <string>

int main(int argc, char* argv[])
{
    if (argc == 4 && std::string(argv[1]) == "keygen") {
        if (!CipherFile::generateKeys(argv[2], argv[3])) {
            std::cerr << "Error: key generation failed\n";
            return 1;
        }
        std::cout << "Key pair generated\n";
        return 0;
    }

    if (argc == 5 && std::string(argv[1]) == "encrypt") {
        if (!CipherFile::encryptFile(argv[2], argv[3], argv[4])) {
            std::cerr << "Error: encryption failed\n";
            return 1;
        }
        std::cout << "File encrypted\n";
        return 0;
    }

    if (argc == 5 && std::string(argv[1]) == "decrypt") {
        if (!CipherFile::decryptFile(argv[2], argv[3], argv[4])) {
            std::cerr << "Error: decryption failed\n";
            return 1;
        }
        std::cout << "File decrypted\n";
        return 0;
    }

    std::cerr << "Usage:\n"
                 << "   " << argv[0] << " keygen <private_key_file>
<public_key_file>\n"
              << "  " << argv[0] << " encrypt <public_key_file> <input_file>
<encrypted_file>\n"
                 << "   " << argv[0] << " decrypt <private_key_file>
<encrypted_file> <output_file>\n";
    return 1;
}

#pragma once

#include <array>
#include <cstdint>
#include <string>
#include <vector>

using Byte = std::uint8_t;
using Bytes = std::vector<Byte>;

struct UInt256 {
    std::array<std::uint64_t, 4> limb{};

    bool zero() const
```

```
        {
            return limb[0] == 0 && limb[1] == 0 && limb[2] == 0 && limb[3] ==
0;
        }

        bool bit(unsigned index) const
        {
            return ((limb[index / 64] >> (index % 64)) & 1U) != 0;
        }

        static int compare(const UInt256& left, const UInt256& right);
        static UInt256 addRaw(const UInt256& left, const UInt256& right, bool&
carry);
        static UInt256 subRaw(const UInt256& left, const UInt256& right);
        static int hexDigit(char value);
        static bool fromHex(const std::string& text, UInt256& value);
        static UInt256 fromBytes(const std::array<Byte, 32>& bytes);
        std::array<Byte, 32> toBytes() const;
        static std::string toHex(const UInt256& value);
        static UInt256 addMod(const UInt256& left, const UInt256& right, const
UInt256& modulus);
        static UInt256 subMod(const UInt256& left, const UInt256& right, const
UInt256& modulus);
        static UInt256 mulMod(const UInt256& left, const UInt256& right, const
UInt256& modulus);
        static UInt256 powMod(UInt256 base, const UInt256& exponent, const
UInt256& modulus);
        static UInt256 subtractSmall(UInt256 value, std::uint64_t amount);
    };

    // Jacobian coordinates: affine x = X/Z^2, affine y = Y/Z^3.
    struct CurvePoint {
        UInt256 x;
        UInt256 y;
        UInt256 z;
        bool infinity = true;
    };

    class EllipticCurve {
    public:
        static const UInt256& field();
        static const UInt256& order();
        static const UInt256& curveA();
        static const UInt256& generatorX();
        static const UInt256& generatorY();
        static CurvePoint pointAtInfinity();
        static CurvePoint affinePoint(const UInt256& x, const UInt256& y);
        static CurvePoint pointDouble(const CurvePoint& point);
        static CurvePoint pointAdd(const CurvePoint& left, const CurvePoint&
right);
        static CurvePoint scalarMultiply(const CurvePoint& point, const
UInt256& scalar);
        static bool toAffine(const CurvePoint& point, UInt256& x, UInt256&
y);
        static bool isOnCurve(const UInt256& x, const UInt256& y);
        static const UInt256& curveB();
        static const UInt256& squareRootExponent();
        static CurvePoint negatePoint(CurvePoint point);
        static bool validPublicKey(const UInt256& x, const UInt256& y);
        static CurvePoint generator();
    };

    #include "EllipticCurve.hpp"
```

```
int UInt256::compare(const UInt256& left, const UInt256& right)
{
    for (int i = 3; i >= 0; --i) {
        if (left.limb[i] < right.limb[i]) return -1;
        if (left.limb[i] > right.limb[i]) return 1;
    }
    return 0;
}

UInt256 UInt256::addRaw(const UInt256& left, const UInt256& right, bool&
carry)
{
    UInt256 result;
    carry = false;
    for (unsigned i = 0; i < 4; ++i) {
        const std::uint64_t first = left.limb[i] + right.limb[i];
        const bool firstCarry = first < left.limb[i];
                    const    std::uint64_t    second    =    first    +
static_cast<std::uint64_t>(carry);
        const bool secondCarry = second < first;
        result.limb[i] = second;
        carry = firstCarry || secondCarry;
    }
    return result;
}

UInt256 UInt256::subRaw(const UInt256& left, const UInt256& right)
{
    UInt256 result;
    bool borrow = false;
    for (unsigned i = 0; i < 4; ++i) {
        const std::uint64_t first = left.limb[i] - right.limb[i];
        const bool firstBorrow = left.limb[i] < right.limb[i];
                    const    std::uint64_t    second    =    first    -
static_cast<std::uint64_t>(borrow);
                     const    bool    secondBorrow    =    first    <
static_cast<std::uint64_t>(borrow);
        result.limb[i] = second;
        borrow = firstBorrow || secondBorrow;
    }
    return result;
}

int UInt256::hexDigit(char value)
{
    if (value >= '0' && value <= '9') return value - '0';
    if (value >= 'a' && value <= 'f') return value - 'a' + 10;
    if (value >= 'A' && value <= 'F') return value - 'A' + 10;
    return -1;
}

bool UInt256::fromHex(const std::string& text, UInt256& value)
{
    if (text.size() != 64) return false;
    value = {};
    for (std::size_t i = 0; i < text.size(); ++i) {
        const int nibble = hexDigit(text[text.size() - 1 - i]);
        if (nibble < 0) return false;
         value.limb[i / 16] |= static_cast<std::uint64_t>(nibble) << ((i
% 16) * 4);
        }
    return true;
}
```

```
UInt256 UInt256::fromBytes(const std::array<Byte, 32>& bytes)
{
    UInt256 value;
    for (unsigned i = 0; i < bytes.size(); ++i) {
        value.limb[(31 - i) / 8] |=
            static_cast<std::uint64_t>(bytes[i]) << (((31 - i) % 8) * 8);
    }
    return value;
}

std::string UInt256::toHex(const UInt256& value)
{
    static constexpr char digits[] = "0123456789abcdef";
    std::string result(64, '0');
    for (unsigned i = 0; i < 32; ++i) {
        const Byte byte = static_cast<Byte>(value.limb[(31 - i) / 8] >>
(((31 - i) % 8) * 8));
        result[i * 2] = digits[byte >> 4];
        result[i * 2 + 1] = digits[byte & 0x0f];
    }
    return result;
}

std::array<Byte, 32> UInt256::toBytes() const
{
    std::array<Byte, 32> result{};
    for (unsigned i = 0; i < result.size(); ++i) {
        result[i] = static_cast<Byte>(limb[(31 - i) / 8] >> (((31 - i) %
8) * 8));
    }
    return result;
}

UInt256 UInt256::addMod(const UInt256& left, const UInt256& right, const
UInt256& modulus)
{
    bool carry = false;
    UInt256 sum = addRaw(left, right, carry);
    if (carry || compare(sum, modulus) >= 0) sum = subRaw(sum, modulus);
    return sum;
}

UInt256 UInt256::subMod(const UInt256& left, const UInt256& right, const
UInt256& modulus)
{
    if (compare(left, right) >= 0) return subRaw(left, right);
    return subRaw(modulus, subRaw(right, left));
}

UInt256 UInt256::mulMod(const UInt256& left, const UInt256& right, const
UInt256& modulus)
{
    UInt256 result;
    UInt256 addend = left;
    for (unsigned bit = 0; bit < 256; ++bit) {
        if (right.bit(bit)) result = addMod(result, addend, modulus);
        if (bit != 255) addend = addMod(addend, addend, modulus);
    }
    return result;
}

UInt256 UInt256::powMod(UInt256 base, const UInt256& exponent, const
UInt256& modulus)
{
```

```
        UInt256 result;
        result.limb[0] = 1;
        for (unsigned bit = 0; bit < 256; ++bit) {
            if (exponent.bit(bit)) result = mulMod(result, base, modulus);
            if (bit != 255) base = mulMod(base, base, modulus);
        }
        return result;
    }

    UInt256 UInt256::subtractSmall(UInt256 value, std::uint64_t amount)
    {
        UInt256 small;
        small.limb[0] = amount;
        return subRaw(value, small);
    }

    CurvePoint EllipticCurve::generator()
    {
        return affinePoint(generatorX(), generatorY());
    }

    const UInt256& EllipticCurve::field()
    {
        static const UInt256 value = [] {
            UInt256 result;
            UInt256::fromHex("ffffff
```

```
const UInt256& EllipticCurve::generatorY()
{
    static const UInt256 value = [] {
        UInt256 result;
        UInt256::fromHex("32879423ab1a0375895786c4bb46e9565fde0b5344766740af268adb32322e5c", result);
        return result;
    }();
    return value;
}

CurvePoint EllipticCurve::pointAtInfinity()
{
    return {};
}

CurvePoint EllipticCurve::affinePoint(const UInt256& x, const UInt256& y)
{
    CurvePoint result;
    result.x = x;
    result.y = y;
    result.z.limb[0] = 1;
    result.infinity = false;
    return result;
}

CurvePoint EllipticCurve::pointDouble(const CurvePoint& point)
{
    if (point.infinity || point.y.zero()) return pointAtInfinity();

    const UInt256& p = field();
    const UInt256 xx = UInt256::mulMod(point.x, point.x, p);
    const UInt256 yy = UInt256::mulMod(point.y, point.y, p);
    const UInt256 yyyy = UInt256::mulMod(yy, yy, p);
    const UInt256 zz = UInt256::mulMod(point.z, point.z, p);
    const UInt256 s = UInt256::mulMod(
        UInt256{{4, 0, 0, 0}},
        UInt256::mulMod(point.x, yy, p), p);
    const UInt256 m = UInt256::addMod(
        UInt256::mulMod(UInt256{{3, 0, 0, 0}}, xx, p),
        UInt256::mulMod(curveA(), UInt256::mulMod(zz, zz, p), p), p);
      const UInt256 x3 = UInt256::subMod(UInt256::mulMod(m, m, p), UInt256::addMod(s, s, p), p);
     const UInt256 eightYyyy = UInt256::mulMod(UInt256{{8, 0, 0, 0}}, yyyy, p);
    const UInt256 y3 = UInt256::subMod(
        UInt256::mulMod(m, UInt256::subMod(s, x3, p), p), eightYyyy, p);
    const UInt256 yPlusZ = UInt256::addMod(point.y, point.z, p);
    const UInt256 z3 = UInt256::subMod(
         UInt256::subMod(UInt256::mulMod(yPlusZ, yPlusZ, p), yy, p), zz, p);

    return CurvePoint{x3, y3, z3, false};
}

CurvePoint EllipticCurve::pointAdd(const CurvePoint& left, const CurvePoint& right)
{
    if (left.infinity) return right;
    if (right.infinity) return left;
    const UInt256& p = field();

    const UInt256 z1z1 = UInt256::mulMod(left.z, left.z, p);
```

```
        const UInt256 z2z2 = UInt256::mulMod(right.z, right.z, p);
        const UInt256 u1 = UInt256::mulMod(left.x, z2z2, p);
        const UInt256 u2 = UInt256::mulMod(right.x, z1z1, p);
        const UInt256 s1 = UInt256::mulMod(left.y, UInt256::mulMod(right.z,
z2z2, p), p);
        const UInt256 s2 = UInt256::mulMod(right.y, UInt256::mulMod(left.z,
z1z1, p), p);

        if (UInt256::compare(u1, u2) == 0) {
            return UInt256::compare(s1, s2) == 0 ? pointDouble(left) :
pointAtInfinity();
        }

        const UInt256 h = UInt256::subMod(u2, u1, p);
        const UInt256 i = UInt256::mulMod(UInt256{{4, 0, 0, 0}},
UInt256::mulMod(h, h, p), p);
        const UInt256 j = UInt256::mulMod(h, i, p);
        const UInt256 r = UInt256::mulMod(UInt256{{2, 0, 0, 0}},
UInt256::subMod(s2, s1, p), p);
        const UInt256 v = UInt256::mulMod(u1, i, p);
        const UInt256 x3 = UInt256::subMod(UInt256::subMod(UInt256::mulMod(r,
r, p), j, p), UInt256::addMod(v, v, p), p);
        const UInt256 y3 = UInt256::subMod(
            UInt256::mulMod(r, UInt256::subMod(v, x3, p), p),
            UInt256::mulMod(UInt256{{2, 0, 0, 0}}, UInt256::mulMod(s1, j, p),
p), p);
        const UInt256 zSum = UInt256::addMod(left.z, right.z, p);
        const UInt256 z3 = UInt256::mulMod(
            UInt256::subMod(UInt256::subMod(UInt256::mulMod(zSum, zSum, p),
z1z1, p), z2z2, p), h, p);
        return CurvePoint{x3, y3, z3, false};
    }

    CurvePoint EllipticCurve::scalarMultiply(const CurvePoint& point, const
UInt256& scalar)
    {
        CurvePoint result = pointAtInfinity();
        CurvePoint addend = point;
        for (unsigned bit = 0; bit < 256; ++bit) {
            if (scalar.bit(bit)) result = pointAdd(result, addend);
            if (bit != 255) addend = pointDouble(addend);
        }
        return result;
    }

    bool EllipticCurve::toAffine(const CurvePoint& point, UInt256& x,
UInt256& y)
    {
        if (point.infinity) return false;
        const UInt256 zInverse = UInt256::powMod(point.z,
UInt256::subtractSmall(field(), 2), field());
        const UInt256 zInverse2 = UInt256::mulMod(zInverse, zInverse,
field());
        const UInt256 zInverse3 = UInt256::mulMod(zInverse2, zInverse,
field());
        x = UInt256::mulMod(point.x, zInverse2, field());
        y = UInt256::mulMod(point.y, zInverse3, field());
        return true;
    }

    bool EllipticCurve::isOnCurve(const UInt256& x, const UInt256& y)
    {
        if (UInt256::compare(x, field()) >= 0 || UInt256::compare(y, field())
>= 0) return false;
```

```
        const UInt256 left = UInt256::mulMod(y, y, field());
        const UInt256 x2 = UInt256::mulMod(x, x, field());
        const UInt256 x3 = UInt256::mulMod(x2, x, field());
        const UInt256 right = UInt256::addMod(
               UInt256::addMod(x3,  UInt256::mulMod(curveA(),  x,  field()),
field()),
            curveB(), field());
        return UInt256::compare(left, right) == 0;
    }

    const UInt256& EllipticCurve::curveB()
    {
        static const UInt256 value = [] {
            UInt256 result;
            UInt256::fromHex("295f9bae7428ed9ccc20e7c359a9d41a22fccd9108e17b
f7ba9337a6f8ae9513", result);
            return result;
        }();
        return value;
    }

    const UInt256& EllipticCurve::squareRootExponent()
    {
        static const UInt256 value = [] {
            UInt256 result;
            UInt256::fromHex("3fffffffffffffffffffffffffffffffffffffffffffff
ffffffffffffff66", result);
            return result;
        }();
        return value;
    }

    CurvePoint EllipticCurve::negatePoint(CurvePoint point)
    {
             if    (!point.infinity    &&    !point.y.zero())    point.y    =
UInt256::subRaw(field(), point.y);
        return point;
    }

    bool EllipticCurve::validPublicKey(const UInt256& x, const UInt256& y)
    {
        if (!isOnCurve(x, y)) return false;
        const CurvePoint key = affinePoint(x, y);
        return scalarMultiply(key, order()).infinity;
    }

    #pragma once

    #include "EllipticCurve.hpp"

    struct CipherBlock {
        CurvePoint c1;
        CurvePoint c2;
    };

    class ElGamal {
    public:
        static constexpr std::size_t BlockSize = 30;

        explicit ElGamal(const CurvePoint& publicKey);
        explicit ElGamal(const UInt256& privateKey);

         bool  encrypt(const  CurvePoint&  message,  CipherBlock&  ciphertext)
const;
```

```
           bool decrypt(const CipherBlock& ciphertext, CurvePoint& message)
const;
            static bool messagePoint(const Byte* data, std::size_t size,
CurvePoint& point);
          static bool pointMessage(const CurvePoint& point, std::array<Byte,
BlockSize>& result);
             static bool generateKeyPair(UInt256& privateKey, CurvePoint&
publicKey);

      private:
          static UInt256 shiftLeft8(const UInt256& value);
          static UInt256 shiftRight8(const UInt256& value);
          static UInt256 bytesToInteger(const Byte* data, std::size_t size);
          static bool randomScalar(UInt256& result);
          CurvePoint publicKey_;
          UInt256 privateKey_;
          bool hasPrivateKey_ = false;
      };

      #include "ElGamal.hpp"

      #include <algorithm>

      #ifndef NOMINMAX
      #define NOMINMAX
      #endif
      #include <windows.h>
      #include <bcrypt.h>

      ElGamal::ElGamal(const CurvePoint& publicKey) : publicKey_(publicKey)
      {
      }

      ElGamal::ElGamal(const UInt256& privateKey)
          : privateKey_(privateKey), hasPrivateKey_(true)
      {
      }

      bool ElGamal::encrypt(const CurvePoint& message, CipherBlock& ciphertext)
const
      {
          if (publicKey_.infinity || message.infinity) return false;
          const CurvePoint generator = EllipticCurve::generator();

          do {
              UInt256 nonce;
              if (!randomScalar(nonce)) return false;
              ciphertext.c1 = EllipticCurve::scalarMultiply(generator, nonce);
                                    const      CurvePoint      shared      =
EllipticCurve::scalarMultiply(publicKey_, nonce);
              ciphertext.c2 = EllipticCurve::pointAdd(message, shared);
                // The container stores finite points only; retry the rare
cancellation.
          } while (ciphertext.c2.infinity);
          return true;
      }

      bool ElGamal::decrypt(const CipherBlock& ciphertext, CurvePoint& message)
const
      {
          if (!hasPrivateKey_ || privateKey_.zero() ||
              UInt256::compare(privateKey_, EllipticCurve::order()) >= 0 ||
              ciphertext.c1.infinity || ciphertext.c2.infinity ||
```

```
                                !EllipticCurve::scalarMultiply(ciphertext.c1,
EllipticCurve::order()).infinity) {
            return false;
        }
                            const          CurvePoint          shared          =
EllipticCurve::scalarMultiply(ciphertext.c1, privateKey_);
                    message      =      EllipticCurve::pointAdd(ciphertext.c2,
EllipticCurve::negatePoint(shared));
        return !message.infinity;
    }

    bool ElGamal::randomScalar(UInt256& result)
    {
        std::array<Byte, 32> bytes{};
        do {
                             if     (BCryptGenRandom(nullptr,     bytes.data(),
static_cast<ULONG>(bytes.size()),
                                  BCRYPT_USE_SYSTEM_PREFERRED_RNG) != 0) {
                return false;
            }
            result = UInt256::fromBytes(bytes);
              }    while    (result.zero()    ||    UInt256::compare(result,
EllipticCurve::order()) >= 0);
        return true;
    }

    bool ElGamal::generateKeyPair(UInt256& privateKey, CurvePoint& publicKey)
    {
        if (!randomScalar(privateKey)) return false;
       publicKey = EllipticCurve::scalarMultiply(EllipticCurve::generator(),
privateKey);
        return !publicKey.infinity;
    }

    UInt256 ElGamal::shiftLeft8(const UInt256& value)
    {
        UInt256 result;
        for (unsigned i = 0; i < 4; ++i) {
            result.limb[i] = value.limb[i] << 8;
            if (i != 0) result.limb[i] |= value.limb[i - 1] >> 56;
        }
        return result;
    }

    UInt256 ElGamal::shiftRight8(const UInt256& value)
    {
        UInt256 result;
        for (unsigned i = 0; i < 4; ++i) {
            result.limb[i] = value.limb[i] >> 8;
            if (i != 3) result.limb[i] |= value.limb[i + 1] << 56;
        }
        return result;
    }

    UInt256 ElGamal::bytesToInteger(const Byte* data, std::size_t size)
    {
        UInt256 result;
        for (std::size_t i = 0; i < size; ++i) {
            result = shiftLeft8(result);
            result.limb[0] |= data[i];
        }
        return result;
    }
```

```
      bool   ElGamal::messagePoint(const   Byte*   data,   std::size_t   size,
CurvePoint& point)
      {
          if (size == 0 || size > BlockSize || data == nullptr) return false;
          // Reversible encoding: x = 256*m + tag, 0 <= tag < 256.
          const UInt256 message = bytesToInteger(data, size);
          UInt256 x = shiftLeft8(message);

          for (unsigned tag = 0; tag < 256; ++tag) {
               x.limb[0] = (x.limb[0] & ~static_cast<std::uint64_t>(0xff)) |
tag;
              if (UInt256::compare(x, EllipticCurve::field()) >= 0) continue;

              const UInt256 x2 = UInt256::mulMod(x, x, EllipticCurve::field());
              const UInt256 x3 = UInt256::mulMod(x2, x, EllipticCurve::field());
               const UInt256 rhs = UInt256::addMod(
                   UInt256::addMod(x3, UInt256::mulMod(EllipticCurve::curveA(),
x, EllipticCurve::field()), EllipticCurve::field()),
                   EllipticCurve::curveB(), EllipticCurve::field());
                            const    UInt256    y    =    UInt256::powMod(rhs,
EllipticCurve::squareRootExponent(), EllipticCurve::field());
                           if     (UInt256::compare(UInt256::mulMod(y,     y,
EllipticCurve::field()), rhs) == 0) {
                   point = EllipticCurve::affinePoint(x, y);
                   return true;
               }
          }
          return false;
      }

      bool  ElGamal::pointMessage(const  CurvePoint&  point,  std::array<Byte,
ElGamal::BlockSize>& result)
      {
          UInt256 x;
          UInt256 y;
          if (!EllipticCurve::toAffine(point, x, y)) return false;
          const UInt256 message = shiftRight8(x);

          // After removing the tag, m must fit in 30 bytes (240 bits).
          if ((message.limb[3] >> 48) != 0) return false;

          const auto bytes = message.toBytes();
           std::copy(bytes.end() - static_cast<std::ptrdiff_t>(result.size()),
bytes.end(), result.begin());
          return true;
      }

      #pragma once

      #include "EllipticCurve.hpp"

      #include <array>
      #include <istream>
      #include <ostream>
      #include <string>
      #include <utility>
      #include <vector>

      class CipherFile {
      public:
            static  bool  generateKeys(const  std::string&  privatePath,  const
std::string& publicPath);
          static bool encryptFile(const std::string& publicPath,
                                  const std::string& inputPath,
```

```
                                    const std::string& outputPath);
          static bool decryptFile(const std::string& privatePath,
                                    const std::string& inputPath,
                                    const std::string& outputPath);

      private:
            static bool readPrivateKey(const std::string& path, UInt256&
privateKey);
            static bool readPublicKey(const std::string& path, CurvePoint&
publicKey);
          static bool parseFields(const std::string& path,
                                  const std::vector<std::string>& names,
                                  std::vector<UInt256>& values);
          static bool writeFields(const std::string& path,
                                     const std::vector<std::pair<std::string,
UInt256>>& fields);

           static bool readBytes(std::istream& input, Byte* data, std::size_t
size);
            static bool writeBytes(std::ostream& output, const Byte* data,
std::size_t size);
          static bool readPoint(std::istream& input, CurvePoint& point);
          static bool writePoint(std::ostream& output, const CurvePoint& point);
          static bool readFile(const std::string& path, Bytes& data);
          static bool writeUInt64(std::ostream& output, std::uint64_t value);
          static bool readUInt64(std::istream& input, std::uint64_t& value);
           static constexpr std::array<char, 8> Magic = {'E', 'C', 'E', 'L',
'G', '1', 0, 0};
      };

      #include "CipherFile.hpp"
      #include "ElGamal.hpp"

      #include <algorithm>
      #include <fstream>
      #include <iterator>
      #include <limits>

      bool CipherFile::readBytes(std::istream& input, Byte* data, std::size_t
size)
      {
                                     input.read(reinterpret_cast<char*>(data),
static_cast<std::streamsize>(size));
                    return      input.good()      ||      input.gcount()      ==
static_cast<std::streamsize>(size);
      }

      bool  CipherFile::writeBytes(std::ostream&  output,  const  Byte*  data,
std::size_t size)
      {
                         output.write(reinterpret_cast<const       char*>(data),
static_cast<std::streamsize>(size));
          return output.good();
      }

      bool CipherFile::readPoint(std::istream& input, CurvePoint& point)
      {
          std::array<Byte, 32> xBytes{};
          std::array<Byte, 32> yBytes{};
          if (!readBytes(input, xBytes.data(), xBytes.size()) ||
              !readBytes(input, yBytes.data(), yBytes.size())) {
              return false;
          }
```

```
            point.x = UInt256::fromBytes(xBytes);
            point.y = UInt256::fromBytes(yBytes);
            point.z = {};
            point.z.limb[0] = 1;
            point.infinity = false;
            return EllipticCurve::isOnCurve(point.x, point.y);
        }

        bool  CipherFile::writePoint(std::ostream&  output,  const  CurvePoint&
point)
        {
            UInt256 x;
            UInt256 y;
            if (!EllipticCurve::toAffine(point, x, y)) return false;
            const auto xBytes = x.toBytes();
            const auto yBytes = y.toBytes();
            return writeBytes(output, xBytes.data(), xBytes.size()) &&
                   writeBytes(output, yBytes.data(), yBytes.size());
        }

        bool CipherFile::readFile(const std::string& path, Bytes& data)
        {
            std::ifstream input(path, std::ios::binary);
            if (!input) return false;
                              data.assign(std::istreambuf_iterator<char>(input),
std::istreambuf_iterator<char>());
            return input.eof() || !input.fail();
        }

        bool CipherFile::writeUInt64(std::ostream& output, std::uint64_t value)
        {
            std::array<Byte, 8> bytes{};
              for  (unsigned  i  =  0;  i  <  bytes.size();  ++i)  bytes[i]  =
static_cast<Byte>(value >> (i * 8));
            return writeBytes(output, bytes.data(), bytes.size());
        }

        bool CipherFile::readUInt64(std::istream& input, std::uint64_t& value)
        {
            std::array<Byte, 8> bytes{};
            if (!readBytes(input, bytes.data(), bytes.size())) return false;
            value = 0;
              for  (unsigned  i  =  0;  i  <  bytes.size();  ++i)  value  |=
static_cast<std::uint64_t>(bytes[i]) << (i * 8);
            return true;
        }

        bool CipherFile::encryptFile(const std::string& publicPath,
                                     const std::string& inputPath,
                                     const std::string& outputPath)
        {
            if (outputPath == publicPath || outputPath == inputPath) return false;

            CurvePoint publicKey;
            Bytes input;
            if (!CipherFile::readPublicKey(publicPath, publicKey) ||
                !readFile(inputPath, input)) {
                return false;
            }

            const ElGamal cipher(publicKey);
            std::ofstream output(outputPath, std::ios::binary | std::ios::trunc);
            if (!output) return false;
```

```
          if (!writeBytes(output, reinterpret_cast<const Byte*>(Magic.data()),
Magic.size()) ||
                                  !writeBytes(output,     reinterpret_cast<const
Byte*>("\x01\x1e\x00\x00"), 4) ||
                !writeUInt64(output, static_cast<std::uint64_t>(input.size())))
{
              return false;
          }

           for (std::size_t offset = 0; offset < input.size(); offset +=
ElGamal::BlockSize) {
             const std::size_t size = std::min(ElGamal::BlockSize, input.size()
- offset);
              CurvePoint message;
             if (!ElGamal::messagePoint(input.data() + offset, size, message))
return false;

              CipherBlock block;
              if (!cipher.encrypt(message, block) ||
                       !writePoint(output,  block.c1)  ||  !writePoint(output,
block.c2)) return false;
          }

          output.flush();
          return output.good();
      }

      bool CipherFile::decryptFile(const std::string& privatePath,
                                   const std::string& inputPath,
                                   const std::string& outputPath)
      {
           if (outputPath == privatePath || outputPath == inputPath) return
false;

          UInt256 privateKey;
          if (!CipherFile::readPrivateKey(privatePath, privateKey)) {
              return false;
          }

          std::ifstream input(inputPath, std::ios::binary);
          if (!input) return false;

          std::array<Byte, 8> magic{};
          if (!readBytes(input, magic.data(), magic.size()) || magic !=
              std::array<Byte, 8>{'E', 'C', 'E', 'L', 'G', '1', 0, 0}) {
              return false;
          }

          std::array<Byte, 4> header{};
          if (!readBytes(input, header.data(), header.size()) ||
              header[0] != 1 || header[1] != ElGamal::BlockSize || header[2] !=
0 || header[3] != 0) {
              return false;
          }

          std::uint64_t originalSize = 0;
          if (!readUInt64(input, originalSize) ||
                                                    originalSize           >
static_cast<std::uint64_t>(std::numeric_limits<std::size_t>::max())) {
              return false;
          }

          const std::size_t size = static_cast<std::size_t>(originalSize);
          const std::size_t blockCount = size / ElGamal::BlockSize +
```

```
                                        (size % ElGamal::BlockSize != 0);
          const auto payloadStart = input.tellg();
          input.seekg(0, std::ios::end);
          const auto payloadEnd = input.tellg();
          constexpr std::size_t encodedBlockSize = 128;
          if (payloadStart < 0 || payloadEnd < payloadStart ||
                  static_cast<std::uint64_t>(payloadEnd  -  payloadStart)  /
encodedBlockSize != blockCount ||
              (payloadEnd - payloadStart) % encodedBlockSize != 0) return false;
          input.seekg(payloadStart);

          const ElGamal cipher(privateKey);
          Bytes output;
          output.reserve(size);

          for (std::size_t block = 0; block < blockCount; ++block) {
              CipherBlock ciphertext;
                 if  (!readPoint(input,  ciphertext.c1)  ||  !readPoint(input,
ciphertext.c2)) return false;

              CurvePoint message;
              if (!cipher.decrypt(ciphertext, message)) return false;
              std::array<Byte, ElGamal::BlockSize> decoded{};
              if (!ElGamal::pointMessage(message, decoded)) return false;

               const std::size_t blockSize = std::min(ElGamal::BlockSize, size
- block * ElGamal::BlockSize);
                           output.insert(output.end(),     decoded.end()     -
static_cast<std::ptrdiff_t>(blockSize), decoded.end());
          }

          Byte trailing = 0;
          if (input.read(reinterpret_cast<char*>(&trailing), 1)) return false;
          if (!input.eof() || output.size() != size) return false;

          std::ofstream result(outputPath, std::ios::binary | std::ios::trunc);
          if (!result) return false;
          if (!writeBytes(result, output.data(), output.size())) return false;
          result.flush();
          return result.good();
      }

      bool   CipherFile::readPrivateKey(const   std::string&   path,   UInt256&
privateKey)
      {
          std::vector<UInt256> fields;
          if (!parseFields(path, {"d"}, fields) || fields[0].zero() ||
              UInt256::compare(fields[0], EllipticCurve::order()) >= 0) return
false;
          privateKey = fields[0];
          return true;
      }

      bool  CipherFile::readPublicKey(const  std::string&  path,  CurvePoint&
publicKey)
      {
          std::vector<UInt256> fields;
          if (!parseFields(path, {"x", "y"}, fields) ||
                 !EllipticCurve::validPublicKey(fields[0],  fields[1]))  return
false;
          publicKey = EllipticCurve::affinePoint(fields[0], fields[1]);
          return true;
      }
```

```
bool CipherFile::parseFields(const std::string& path,
                             const std::vector<std::string>& names,
                             std::vector<UInt256>& values)
{
    std::ifstream input(path);
    if (!input) return false;
    values.clear();
    std::string line;
    for (const auto& name : names) {
        if (!std::getline(input, line)) return false;
        if (!line.empty() && line.back() == '\r') line.pop_back();
         if (line.compare(0, name.size() + 1, name + "=") != 0) return
false;
        UInt256 value;
       if (!UInt256::fromHex(line.substr(name.size() + 1), value)) return
false;
        values.push_back(value);
    }
    return !std::getline(input, line);
}

bool CipherFile::writeFields(const std::string& path,
                                const std::vector<std::pair<std::string,
UInt256>>& fields)
{
    std::ofstream output(path, std::ios::trunc);
    if (!output) return false;
     for (const auto& field : fields) output << field.first << '=' <<
UInt256::toHex(field.second) << '\n';
    output.flush();
    return output.good();
}

bool CipherFile::generateKeys(const std::string& privatePath, const
std::string& publicPath)
{
    if (privatePath == publicPath) return false;

    UInt256 privateKey;
    UInt256 publicX;
    UInt256 publicY;
    CurvePoint publicKey;
    if (!ElGamal::generateKeyPair(privateKey, publicKey) ||
        !EllipticCurve::toAffine(publicKey, publicX, publicY)) {
        return false;
    }

    return writeFields(privatePath, {{"d", privateKey}}) &&
           writeFields(publicPath, {{"x", publicX}, {"y", publicY}});
}
```